

The Call Center Customer Care (C4) Case Study provided as an appendix to this assignment presents an initial high-level (“Level 1”) architectural breakdown for the system used by a large telecommunications company. This system comprises several subsystems, one of which is C4 itself.

1. The lectures and readings have begun discussing the architectural design process, including the selection of architectural styles. Not all styles are applicable to all systems, and any choice of style will involve making trades on various system attributes. Pick any two styles that you have read about and design two different architectures for the C4 system, one that adheres to each style. In order to apply a style, you will need to create a detailed architectural breakdown (a “Level 2” architectural breakdown) for C4. In other words, “expand” the C4 box shown in the figure on page 1 in the Case Study into an architecture. It is, of course, difficult to decide on the exact degree of detail to be provided in a “Level 2” architecture, such as the one required for this assignment. Make sure to show all the connectors inside the C4 architecture as well as those that interconnect C4 to the other parts of the system. Moreover, you should graphically distinguish different types of connectors used in your design. Also, there is no such thing as the “correct” or “optimal” architecture. However, as a granularity guideline, your decomposition of C4 should consist of no less than 20 and no more than 30 distinct components.¹ You are not required to select a style from the course textbook, but you must let us know what style you are attempting to apply and provide a reference if the style is not one from the course text.² Submit one diagram for each architecture you design.

Answer: The Diagram for this case study is attached separately for this.

I have attached 2 Diagrams:

- 1) Architecture A - **Layered + Client-Server** (24 components) - c4_architecture_a.png
- 2) Architecture B - **Event-Driven Publish/Subscribe with CQRS + Saga** (24 components) - c4_architecture_b.png

2. Give a brief rationale for your architecture: why did you select the styles that you did? Suggest two pros and cons of each architectural style and weigh them. We will not grade you based on how accurately you apply each style so much as your rationale for selecting a

particular style and understanding its limitations. Please limit your answer to 2 paragraphs for each style (i.e., a total of 4 paragraphs).

Answer:

Architecture A - Layered + Client-Server (24 components):

--> Why this style?

This is a layered architecture with a clear client-server boundary that centralizes business rules in the Application/Domain layer and data integrity in the Data layer. This is a solid fit for C4's OLTP nature: agents carry out "events → tasks → activities" with heavy validation (service availability, completeness checks, data integrity) and strong coupling to corporate online databases. Placing integration concerns (NOSS/Downstream) behind an Integration layer encapsulates external protocols and asynchronous patterns, keeping the core logic simple.

--> Pros/Cons and tradeoffs:

Pros:

- (1) **Maintainability & understandability** - traceable call paths, responsibilities are obvious per layer;
- (2) **Data integrity** - one DAL mediates updates to the online DBs, enabling consistent validation and transaction handling.

Cons:

- (1) **Scalability flexibility** is more limited - coarse-grained services may become bottlenecks;
- (2) **Asynchrony modeling** is less natural - event-heavy integrations (NOSS callbacks, downstream notifications) require extra adapters and trackers to avoid leaking async complexity into synchronous domains.

Tradeoff: We get predictable control flow and strong validation at the cost of more glue code around asynchronous edges.

Architecture B - Event-Driven Publish/Subscribe with CQRS + Saga (24 components):

--> Why this style?

C4 must handle heavy concurrency (up to ~400 reps), long-running provisioning and asynchronous integrations (NOSS request/reply with later callbacks; downstream notifications). An event-driven backbone matches these realities: commands trigger domain changes; domain events drive read-model updates, cross-component reactions, and integrations. Sagas coordinate multi-step flows across NOSS and downstream systems without blocking front-end calls, improving responsiveness and throughput.

--> Pros/Cons and tradeoffs:

Pros:

- (1) **Scalability & elasticity** - event buses and read models scale horizontally;
- (2) **Integration fit** - async callbacks and P/S map naturally to inbound/outbound event streams.

Cons:

(1) Consistency & traceability are harder - distributed updates and eventual consistency complicate correctness and debugging;

(2) Operational complexity - more moving parts (buses, projections, sagas) require mature monitoring and ops.

Tradeoff: We gain throughput and extensibility but must invest in robust tracing, idempotency, and conflict policies.

3. Compare each of your architectures: give an example or two of a system property or requirement described in the Case Study that one of your architectures addresses in a superior manner. Be sure to not only name the property or requirement but also explain how each architecture addresses the property or requirement and give the rationale for why you think one system is superior to the other. Please limit your answer to 2 paragraphs

Answer:

Data integrity during OLTP updates (customer profile, orders, tickets) is generally **better in Architecture A**. Centralized validation and a single DAL control write paths to corporate online DBs, making it straightforward to enforce multi-table constraints and transactional rules synchronously during a call. In A, the Task Orchestrator + Validation Service + DAL ensure “complete activities/tasks,” service availability checks, and final configuration integrity before committing.

Handling asynchronous integrations (NOSS request -> later callback; downstream notifications; publish new line to 911 within SLA) is **superior in Architecture B**. The **Event Bus**, **Inbound Listeners**, and **Saga** make callbacks first-class events, which can be correlated, retried, and reacted to by multiple subscribers (e.g., read models, conflict policy, appointment projection) without coupling. New downstream consumers (e.g., additional regulatory feeds) can subscribe without changing core logic.

4. Since C4 is a vast system with many different, possibly conflicting, requirements, your architecture may only directly address a subset. To demonstrate this, for one of your architectures, select three of the key architectural challenges and requirements (listed in the C4 Case Study appendix) and argue and discuss how your architecture DOES NOT support them in an acceptable fashion. Please limit your answer to 2 paragraphs.

Answer: I will be talking about Limitations of Architecture B for this question (3 C4 challenges):

- **Strong, immediate consistency across corporate online DB tables** during complex updates is challenging in B. Commands produce events and projections update asynchronously; under load, read models may briefly lag, which can confuse agents who expect “what I just did” to be visible immediately everywhere.
- **Conflict handling** when **customer requests** collide with **downstream provider notifications** is subtle. While a Conflict Resolution Policy can react to events, ensuring **ordering**, **deduplication**, and **idempotent** application across multiple handlers is non-trivial.
- **End-to-end traceability** of one customer interaction (from call → NOSS requests → callbacks → downstream notifications) is harder. With many asynchronous hops, without strict request-ID propagation and sagas that publish state transitions, debugging and compliance audits can become painful.

5. Modify one of your architectures to address the shortcomings discussed in the preceding question. Can you do so without violating the chosen architectural style? Give the rationale for whether you think this change is appropriate for the system. Please limit your answer to 1 paragraph and a new diagram.

Answer: (The diagram is attached to the zip named: c4_architecture_b_modified.png)

Change (keeps the event-driven style): We will add a **Transactional Outbox** to each command service, a **Global Policy Enforcer** (idempotency, ordering), and a **Trace Correlator** that **propagates a Request-ID** across commands, events, and integrations. Commands **commit changes and outbox messages atomically**; a relay publishes from the outbox to the Event Bus reliably (no missing/duplicate events). The Policy Enforcer ensures idempotent event handling and defines conflict-resolution tie-breaks (e.g., “customer-initiated change overrides if timestamp < T unless carrier notification has higher authority flag”). The Trace Correlator stamps and propagates headers so every event/log line is joinable. **Rationale:** This preserves the **event-driven style** while hardening **consistency** (atomic write+publish), **conflict determinism** (global policy), and **observability** (Request-ID span across hops). It’s appropriate here because C4 inherently benefits from async scaling, but needs stronger guarantees and transparency at integration boundaries.

6. Add a capability to both your architectures to log interactions between C4 and Downstream Systems. Logging should be enabled only when there are less than 100 active users interacting with the system concurrently. How will you change each architecture in response to this new requirement? Which architecture can be changed more easily? Why?

Please limit your answer to 2 paragraphs and, if you find it necessary, one diagram for each of your architectures.

Answer:

In Architecture A (Layered): introduce an **Active-User Monitor** (reads session counts from Session Gateway/Telemetry) and a **Traffic-Aware Logger in the Integration layer**. Both **Downstream Publisher** and **Downstream Event Handler** route their interaction records through this logger. The logger checks the current active-user count (via a lightweight in-memory cache, refreshed every few seconds) and **conditionally forwards** the log to the Audit & Compliance Logger only below the threshold. Technically, this is an **Arbitrator connector** in front of the logger that gates calls based on load. This change is localized (Integration/Data layers) and keeps the rest of the application unchanged.

In Architecture B (Event-Driven): add a **Conditional Logging Subscriber** to the **Event Bus** that subscribes to *DownstreamInteraction.events** (emitted by Downstream Outbound Publisher and Inbound Listener). It queries an **Active-User Monitor** (fed by Telemetry) and **only writes** to the Audit Sink when the threshold is met. Because logging becomes just another subscriber, no producers change, and the behavior can be toggled or scaled independently. This is arguably **easier in B** - we plug in/out a subscriber instead of interposing logic into synchronous call paths.

Which is easier to change? Architecture B. The event-driven model lets us attach a small subscriber and a shared monitor; producers stay untouched. In A, we need a conditional gate in the I/O path of the integration calls (still simple, but more invasive to the “hot path”).